

TP 4 — API métier avec base de données

FastAPI · PyTest · TDD · Git

Objectifs du TP

Dans ce TP, vous allez concevoir et développer une **API REST** en Python en appliquant une **démarche TDD (Test Driven Development)**.

À l'issue du TP, vous devrez être capables de :

- Concevoir une API FastAPI avec **règles métier**
 - Mettre en place une **base de données relationnelle**
 - Écrire des **tests automatisés avec PyTest**
 - Appliquer une démarche **TDD complète**
 - Comprendre les enjeux des **tests avec base de données**
 - Expliquer l'intégration des tests dans une **GitHub Action**
-

Consignes importantes

- **Aucun code métier ne doit être écrit avant les tests**
 - Tous les comportements attendus doivent être **définis par des tests**
 - Les tests doivent être :
 - automatisés
 - reproductibles
 - indépendants les uns des autres
 - Le projet doit être **versionné avec Git**
-

Sujet : API de gestion de commandes clients

Vous devez développer une **API de gestion de commandes** pour une petite entreprise.

L'API s'appuie sur :

- **FastAPI**
 - **Une base de données relationnelle**
 - **PyTest** pour les tests automatisés
-

Modèle métier

Entité Client

- id
- nom
- email

Entité Commande

- id
 - client_id
 - montant
 - statut
- (valeurs possibles : *en_attente*, *validée*, *refusée*)
-

Endpoints à implémenter

Créer une commande

POST /orders

Règles métier :

- Le client doit exister
- Le montant doit être strictement positif
- Le statut initial est toujours *en_attente*

Valider une commande

POST /orders/{id}/validate

Règles métier :

- Une commande ne peut être validée que si :
 - son montant est $\leq 5\,000$ €
- Une commande validée ou refusée ne peut plus changer de statut

Lister les commandes d'un client

GET /clients/{id}/orders

Règles métier :

- Retourner toutes les commandes du client

- Trier les commandes par date de création
 - Si le client n'existe pas → erreur
-

Travail attendu sur les tests

Vous devez **appliquer une démarche TDD stricte**.

Types de tests attendus

1. **Tests unitaires métier**
 - Validation du montant
 - Transitions de statut
 - Cas d'erreur métier
 2. **Tests de persistance**
 - Création des clients et commandes en base
 - Lecture après écriture
 - Intégrité des relations
 3. **Tests d'API**
 - Codes HTTP
 - Contenu des réponses
 - Cohérence avec les règles métier
-

Étapes

Étape 1 — Écriture des tests métier

- Écrire les tests PyTest pour les règles métier
- Aucun accès HTTP
- Aucun accès à la base à ce stade

Étape 2 — Conception de la base de données

- Définir le schéma
- Identifier les relations
- Justifier vos choix

Étape 3 — Tests avec base de données

- Tester la création et la lecture des données
- Garantir l'isolement entre les tests
- Expliquer votre stratégie de nettoyage

Étape 4 — Tests d'API

- Tester les 3 endpoints
- Vérifier les réponses HTTP
- Vérifier les erreurs

Étape 5 — Implémentation finale

- Implémenter le code nécessaire pour faire passer les tests
 - Refactoriser si nécessaire **sans casser les tests**
-

Tests automatisés avec base de données (attendus et bonnes pratiques)

Dans ce TP, une partie des tests devra vérifier le comportement de votre application **avec une base de données**. On parle ici de **tests d'intégration** (ou "tests de persistance"), car ils valident que l'écriture et la lecture en base fonctionnent réellement.

Ce que vous devez tester (minimum attendu)

Vos tests doivent couvrir, au minimum :

- **Insertion** d'un client et vérification qu'il existe bien en base
- **Insertion** d'une commande liée à un client (clé étrangère)
- **Lecture** des commandes d'un client et vérification :
 - du nombre de commandes retournées
 - du tri par date de création
- **Règles de cohérence** :
 - impossible de créer une commande pour un client inexistant
 - gestion correcte des erreurs (ex. client non trouvé)

Pourquoi c'est important ?

Ces tests permettent de détecter des erreurs qui n'apparaissent pas dans les tests unitaires, par exemple :

- mauvais schéma / mauvaise relation entre tables
- erreurs de transaction ou de session
- données non persistées ou mal récupérées
- tri incorrect, filtre incorrect, requête incorrecte

Isolement : règle obligatoire

Les tests ne doivent **jamais dépendre** d'une base déjà remplie.
Chaque test doit pouvoir être exécuté seul, n'importe quand, sur une machine vierge.

Vous devez donc garantir que :

- les données créées par un test **ne polluent pas** les autres tests
- les tests sont **reproductibles**
- l'ordre d'exécution des tests **n'a aucune importance**

Stratégies possibles

- **Base SQLite en mémoire** (rapide, idéale pour les tests)
- **Base SQLite fichier dédiée aux tests** (créée/supprimée à chaque test ou session)
- **Transactions + rollback** après chaque test (quand c'est possible)
- **Recréation du schéma** avant chaque test (drop/create)

Exemple : tests PyTest avec base de données (illustration)

Cet exemple est fourni à **titre illustratif** pour montrer **comment tester une base de données** avec PyTest.

Fixture PyTest pour une base de test isolée

```
import pytest

from sqlalchemy import create_engine

from sqlalchemy.orm import sessionmaker

@pytest.fixture

def db_session():

    # Base SQLite en mémoire (recréée à chaque test)

    engine = create_engine("sqlite:///memory:")

    SessionLocal = sessionmaker(bind=engine)

    # Création du schéma (tables)

    from app.models import Base

    Base.metadata.create_all(engine)

    session = SessionLocal()

    try:

        yield session

    finally:
```

```
session.close()
```

👉 Cette fixture garantit que :

- chaque test démarre avec une base **vide**
- aucune donnée ne persiste entre les tests

Exemple de test : création et lecture d'un client

```
def test_create_and_read_client(db_session):
```

```
    from app.models import Client
```

```
    client = Client(
```

```
        nom="Dupont",
```

```
        email="dupont@test.com"
```

```
    )
```

```
    db_session.add(client)
```

```
    db_session.commit()
```

```
    result = db_session.query(Client).first()
```

```
    assert result is not None
```

```
    assert result.nom == "Dupont"
```

```
    assert result.email == "dupont@test.com"
```

🎯 Ce test vérifie :

- l'écriture en base
- la lecture immédiate
- la cohérence des données

Exemple de test : relation client / commande

```
def test_order_is_linked_to_client(db_session):
```

```
    from app.models import Client, Order
```

```
    client = Client(nom="Martin", email="martin@test.com")
```

```
    db_session.add(client)
```

```
    db_session.commit()
```

```
order = Order(
    client_id=client.id,
    montant=1200,
    statut="en_attente"
)

db_session.add(order)
db_session.commit()

orders = db_session.query(Order).filter_by(client_id=client.id).all()

assert len(orders) == 1

assert orders[0].montant == 1200
```

🎯 Ce test valide :

- la relation entre entités
- l'intégrité des clés étrangères
- la persistance correcte

Bonus — Intégration continue avec GitHub Actions (optionnel)

En complément du TP, vous pouvez mettre en place une **intégration continue (CI)** à l'aide de **GitHub Actions** afin d'exécuter automatiquement les tests PyTest de votre projet.

Ce bonus vise à reproduire un **contexte professionnel réel** : chaque modification du code déclenche une vérification automatique de sa qualité.

Travail attendu

- Configurer une GitHub Action qui :
 - se déclenche automatiquement lors d'un **push** sur le dépôt
 - installe l'environnement Python nécessaire
 - exécute l'ensemble des tests PyTest
 - gère correctement la base de données utilisée pour les tests
 - échoue si au moins un test ne passe pas